

Multiple Linear Regression Analysis

Marketing Sales Data Analysis

This notebook analyzes a multi-channel marketing dataset using Multiple Linear Regression to understand how different marketing channels jointly affect business outcomes.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_squared_error

# Set visualization style
sns.set_style('whitegrid')
plt.rcParams['figure.figsize'] = (12, 6)
```

1. Data Loading and Exploration

```
# Load the dataset
df = pd.read_csv('marketing_sales_data.csv')

# Display the first 5 rows of the DataFrame
print('First 5 rows of the dataset:')
display(df.head())

print(f'\nDataset shape: {df.shape}')
```

```
# Check for missing values
print('Missing values per column:')
print(df.isnull().sum())

# Display data types and non-null values
print('\nDataFrame Info:')
df.info()

# Display descriptive statistics for numerical columns
print('\nDescriptive statistics:')
display(df.describe())
```

2. Data Preprocessing and Encoding

```
# Perform one-hot encoding for categorical variables 'TV' and 'Influencer'
df_encoded = pd.get_dummies(df, columns=['TV', 'Influencer'], drop_first=True)

# Display the first 5 rows of the encoded DataFrame
print('DataFrame after one-hot encoding:')
display(df_encoded.head())

print(f'\nEncoded columns: {list(df_encoded.columns)}')
```

3. Multicollinearity Analysis

```
# Identify independent variables for VIF calculation
X = df_encoded.drop('Sales', axis=1)

# Convert boolean columns to integer (0 or 1) for VIF calculation
X = X.astype(int)

# Calculate the correlation matrix
print('Correlation Matrix of Independent Variables:')
correlation_matrix = X.corr()
display(correlation_matrix)

# Visualize correlation matrix
plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', center=0,
            square=True, linewidths=1, cbar_kws={"shrink": 0.8})
plt.title('Correlation Matrix of Independent Variables')
plt.tight_layout()
plt.show()

# Calculate VIF for each independent variable
vif_data = pd.DataFrame()
vif_data['feature'] = X.columns
vif_data['VIF'] = [variance_inflation_factor(X.values, i) for i in range(len(X.columns))]

print('\nVariance Inflation Factor (VIF):')
vif_sorted = vif_data.sort_values(by='VIF', ascending=False)
display(vif_sorted)

print('\nVIF Interpretation:')
```

```
print('VIF < 5: Low multicollinearity (acceptable)')
print('VIF >= 5: High multicollinearity (concerning)')
```

4. Multiple Linear Regression Model

```
# Prepare the data for modeling
X_with_const = sm.add_constant(X)
y = df_encoded['Sales']

# Build the OLS regression model
model = sm.OLS(y, X_with_const).fit()

# Display model summary
print(model.summary())
```

5. Model Evaluation and Diagnostics

```
# Extract key metrics
r_squared = model.rsquared
adj_r_squared = model.rsquared_adj
f_statistic = model.fvalue
f_pvalue = model.f_pvalue

print('Model Performance Metrics:')
print(f'R-squared: {r_squared:.4f}')
print(f'Adjusted R-squared: {adj_r_squared:.4f}')
print(f'F-statistic: {f_statistic:.4f}')
print(f'F-statistic p-value: {f_pvalue:.4e}')

# Get predictions
y_pred = model.predict(X_with_const)
residuals = model.resid

# Calculate RMSE
rmse = np.sqrt(mean_squared_error(y, y_pred))
print(f'\nRoot Mean Squared Error (RMSE): {rmse:.4f}')
```

6. Residual Analysis and Assumptions

```
# Create diagnostic plots
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Residuals vs Fitted
axes[0, 0].scatter(y_pred, residuals, alpha=0.6)
axes[0, 0].axhline(y=0, color='r', linestyle='--')
axes[0, 0].set_xlabel('Fitted Values')
axes[0, 0].set_ylabel('Residuals')
axes[0, 0].set_title('Residuals vs Fitted Values')

# Q-Q plot
sm.qqplot(residuals, line='45', ax=axes[0, 1])
axes[0, 1].set_title('Q-Q Plot')

# Histogram of residuals
axes[1, 0].hist(residuals, bins=30, edgecolor='black', alpha=0.7)
axes[1, 0].set_xlabel('Residuals')
axes[1, 0].set_ylabel('Frequency')
axes[1, 0].set_title('Distribution of Residuals')

# Scale-Location plot
standardized_residuals = residuals / residuals.std()
axes[1, 1].scatter(y_pred, np.sqrt(np.abs(standardized_residuals)), alpha=0.6)
axes[1, 1].set_xlabel('Fitted Values')
axes[1, 1].set_ylabel('Sqrt(|Standardized Residuals|)')
axes[1, 1].set_title('Scale-Location Plot')

plt.tight_layout()
plt.show()

print('Residual Analysis Summary:')
print(f'Mean of residuals: {residuals.mean():.4e}')
print(f'Std of residuals: {residuals.std():.4f}')
```

7. Model Interpretation and Recommendations

```
# Coefficient interpretation
coefficients = model.params
pvalues = model.pvalues

coef_df = pd.DataFrame({
    'Coefficient': coefficients,
    'P-value': pvalues,
    'Significant': ['Yes' if p < 0.05 else 'No' for p in pvalues]
})

print('Model Coefficients and Significance:')
display(coef_df)

print('\nInterpretation:')
print('Coefficients represent the change in Sales for a one-unit increase in each variable,')
```

```
print( noiding all other variables constant.' )
print('\nSignificant variables (p < 0.05) have a statistically meaningful impact on Sales.')
```

8. Visualizations and Insights

```
# Plot actual vs predicted
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.scatter(y, y_pred, alpha=0.6)
plt.plot([y.min(), y.max()], [y.min(), y.max()], 'r--', lw=2)
plt.xlabel('Actual Sales')
plt.ylabel('Predicted Sales')
plt.title('Actual vs Predicted Sales')
plt.grid(True, alpha=0.3)

# Coefficient importance
plt.subplot(1, 2, 2)
coef_sorted = coef_df.drop('const').sort_values('Coefficient')
colors = ['red' if x == 'No' else 'green' for x in coef_sorted['Significant']]
plt.barh(coef_sorted.index, coef_sorted['Coefficient'], color=colors)
plt.xlabel('Coefficient Value')
plt.title('Feature Importance (Coefficients)')
plt.legend(['Not Significant', 'Significant'], loc='best')
plt.grid(True, alpha=0.3, axis='x')

plt.tight_layout()
plt.show()
```

9. Conclusions and Business Recommendations

Key Findings:

- 1. **Model Fit:** The model's R-squared value indicates how well marketing variables explain sales variations.
- 2. **Multicollinearity:** VIF analysis shows which variables are highly correlated.
- 3. **Significant Predictors:** Only variables with p-values < 0.05 significantly impact sales.
- 4. **Residual Assumptions:** Check if residuals are normally distributed and homoscedastic.

Recommendations for Marketing Budget Allocation:

- **Focus on high-impact channels:** Allocate more budget to variables with positive, significant coefficients.
- **Monitor multicollinearity:** High VIF values suggest channels are correlated; consider their combined effect.
- **Data-driven decisions:** Use coefficient magnitudes to determine relative investment in each channel.
- **Validate assumptions:** Ensure model assumptions hold before making critical business decisions.